OrchestML: Continuous Integration Tool for Automated Machine Learning Service Compositions

Arogya Kharel
School of Computing
KAIST
Daejeon, South Korea
akharel@kaist.ac.kr

Xiangchi Song
School of Computing
KAIST
Daejeon, South Korea
xcsong@kaist.ac.kr

In-Young Ko
School of Computing
KAIST
Daejeon, South Korea
iko@kaist.ac.kr

***Abstract*—We present OrchestML, a prototype tool for continuous integration of machine learning service pipelines. Modern ML systems increasingly depend on composing specialized services, yet identifying, integrating, and maintaining them across heterogeneous repositories remains challenging. OrchestML addresses this by (i) discovering services through cross-repository semantic search, (ii) generating executable blueprints via a staged LLM-powered pipeline, and (iii) adapting deployed pipelines through performance monitoring and automated recomposition. An interactive web interface enables users to explore composition alternatives, validate service dependencies through graph visualization, and deploy pipelines with integrated monitoring. A case study in robotic navigation shows that OrchestML can automatically detect performance degradation and trigger recomposition, improving execution efficiency by 20%. OrchestML is open sourced at https://github.com/gurkhaman/OrchestML, and the demo video is provided at https://youtu.be/4Z5YMqJU1TA.**

***Index Terms*—machine learning service composition, large language models, automated software integration, adaptive systems, service orchestration**

I. Introduction

Modern ML-enabled systems increasingly require compositions of specialized services to achieve complex functionality [1], [2]. Consider developing an autonomous delivery robot: such a system typically requires computer vision for object recognition, speech processing for command interpretation, SLAM for localization, and path planning for navigation. While repositories like Hugging Face [1], Robot Operating System (ROS) Index [2], and GitHub provide extensive service catalogs, identifying and integrating compatible components remains a significant engineering challenge.

This integration complexity stems from three key factors. First, services are distributed across heterogeneous repositories with varying documentation standards and terminology. Second, composing services from different ecosystems requires careful consideration of interface compatibility, data transformations, and execution dependencies. Third, deployed compositions require ongoing maintenance as services evolve and operational conditions change.

Recent research has explored automated approaches to service composition. HuggingGPT [3] demonstrates effective task decomposition and model chaining within the Hugging Face ecosystem, though it does not address cross-repository discovery or runtime adaptation. LLM4Workflow [4] generates structured API compositions but relies on pre-curated service knowledge base, which may limit applicability to dynamic service ecosystems. These approaches provide valuable foundations but do not fully address the end-to-end composition lifecycle from discovery through maintenance.

We present OrchestML, an LLM-based composition system that address these challenges through three integrated components. First, cross-repository service discovery addresses the semantic gap between different service ecosystems. Modern ML services exist across multiple ecosystems, each with distinct documentation practices. For example, a navigation task might be described as "indoor localization" in one repository and "SLAM-based mapping" in another. OrchestML embeds service descriptions from diverse sources, including README files, API documentation, and package metadata into a unified vector space, enabling discovery based on functional requirements rather than exact keyword matches.

Second, systematic composition generation handles the complexity of reasoning about multiple constraints simultaneously. Service composition requires balancing functional requirements, interface compatibility, and execution dependencies. OrchestML employs a staged pipeline that progressively refines requirements into executable blueprints. This approach distributes the reasoning task across specialized phases—requirement analysis, task decomposition, and service mapping—improving accuracy while maintaining traceability.

Third, performance-driven adaptation maintains composition quality in production environments. Deployments face evolving conditions that can degrade performance over time. OrchestML incorporates a monitoring framework that detects anomalies and triggers automated recomposition. By augmenting the composition pipeline with performance context, the system generates alternative compositions that address identified issues while preserving functional requirements.

Our approach builds on the observation that service descriptions contain rich semantic information about capabilities, interfaces, and usage patterns. By treating these descriptions as comprehensive service representations, we enable LLMs to reason about composition without requiring direct API access

[1]https://huggingface.co/docs/hub/en/index
[2]https://index.ros.org/?search_packages=true

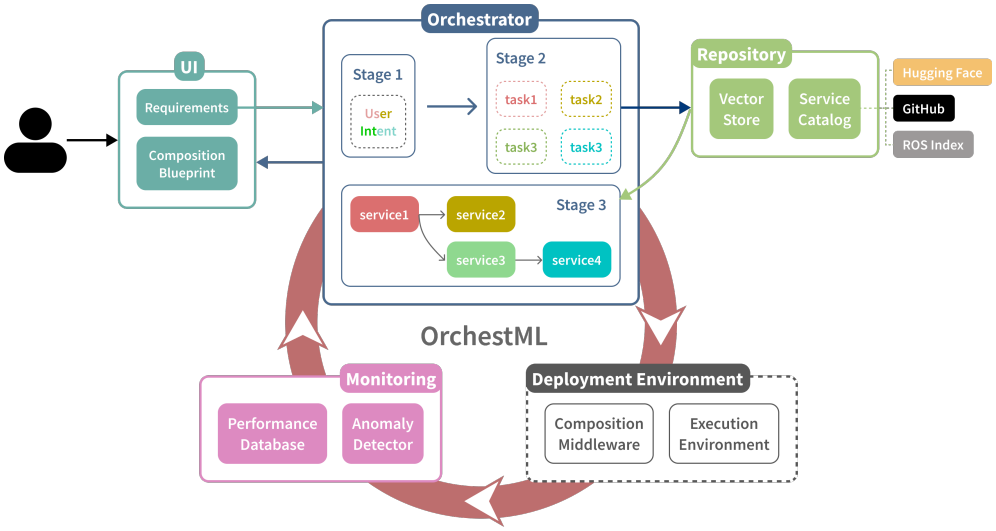


Fig. 1: Overview of OrchestML

or pre-defined integration templates.

We evaluate OrchestML through a case study in robotic navigation, where the system successfully discovered services across ROS Index and GitHub, generated a functional composition blueprint, and automatically adapted the composition when monitoring detected performance degradation. The recomposed pipeline achieved a 20% improvement in navigation time while maintaining safety constraints. The primary contributions of this work are:

1) A unified approach to cross-repository service discovery that leverages semantic similarity to bridge terminology gaps between different service repositories.
2) A staged composition pipeline that systematically transforms natural language requirements into executable service blueprints with explicit dependency management.
3) A continuous adaptation framework that maintains composition quality through performance monitoring and automated recomposition.

II. OrchestML

OrchestML addresses the challenges of automated ML service composition through a modular architecture (Fig. 1). At its core, the **orchestrator module** coordinates a three-stage LLM-powered composition pipeline that incrementally transforms user requirements into executable service blueprints. A **repository module** maintains a vector database of service descriptions aggregated from Hugging Face, ROS Index, and GitHub. A **monitoring module** observes deployed compositions and triggers automated recomposition when performance degradation is detected. Finally, a **web interface** lets users submit requirements and inspect generate blueprints through an interactive graph-based view.

A. Service Repository & Semantic Discovery

The repository module enables cross-repository discovery by embedding heterogeneous service descriptions into a unified semantic space. It treats textual descriptions (e.g., README files, structure metadata, documentation) as proxies for service capabilities, allowing the LLM to reason about functionality without direct integration.

Each service contributes structured metadata, documentation chunks, and code signatures if available. The repository module parses these documents and embeds them using OpenAIEmbeddings (*text-embedding-ada-002-v2*, 1536 dimensions). The resulting vectors are stored in a ChromaDB vector store, which supports similarity-based retrieval using cosine distance. Each query to the repository module retrieves the Top-3 most similar documents.

When the orchestrator decomposes user goals into tasks, the system retrieves semantically aligned services. For example, a task described as "*autonomous indoor navigation*" retrieves services documented as "*Nav2 path planning*," "*SLAM localization*," or "*obstacle avoidance algorithms*".

B. LLM-Powered Composition Pipeline

OrchestML refines natural language requirements into executable composition blueprints via a three-stage pipeline (Fig. 2) that progressively narrows the reasoning space from broad user goals to concrete task-service assignments.

Service composition requires reasoning across interdependent tasks, including understanding user intent, decomposing objectives into executable steps, and mapping those steps to available services. Our three-stage pipeline progressively narrows the reasoning scope, allowing each stage to focus on a well-defined analytical objective. We follow prior work demonstrating that decomposing reasoning into intermediate steps improve LLM reliability, applying chain-of-thought prompting and adding validation checkpoints between stages for error recovery [5]–[7].

1) Stage 1 - Requirement Analysis: The system performs requirement analysis, extracting structured information from natural language user requirements. The system identifies

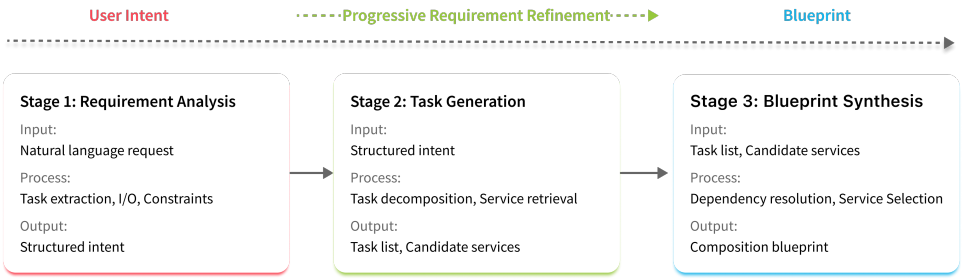


Fig. 2: LLM reasoning pipeline.

core tasks, domains, objectives, I/O, success criteria, and constraints. This analysis establishes the foundation for subsequent stages by creating a formal representation of user intent. Stage 1 in Fig. 2 illustrates this processed user intent.

2) Stage 2 - Task Generation and Retrieval: The system decomposes objectives into atomic tasks and retrieves semantically aligned service candidates from the repository module. Each atomic task represents a single, well-defined functionality that can be mapped to available services (Fig. 2 Stage 2). The system retrieves multiple candidates per task, maintaining alternatives that enable flexibility in the final blueprint synthesis phase.

3) Stage 3 - Blueprint Synthesis: The system synthesizes the composition blueprint by resolving dependencies, determining execution order, and selecting specific services from candidates. Candidate services are combined with task requirements to produce composition blueprints that define task order, service assignments, dependencies, and execution parameters, along with system confidence scores for each alternative. Stage 3 in Fig. 2 shows the blueprint structure.

C. Performance Monitoring & Adaptation Framework

ML service compositions inevitably experience performance drift due to changing operational conditions, service degradation, or evolving requirements. Our automated adaptation framework address this maintenance challenge by systematically detecting performance issues and generating improved compositions using the same LLM-guided reasoning pipeline, creating a closed-loop optimization system that reduces the need for manual expert intervention.

The monitoring framework collects performance metrics from deployed compositions through configurable endpoints. Metrics may include latency, accuracy, resource usage, or task-specific success rates, depending on the application domain. These measurements are stored in time-series database (e.g., TimescaleDB[3]) to enable historical analysis and establishment of rolling baselines.

For anomaly detection, we adopt a simple but extensible statistical approach. Current implementation uses z-score analysis (2-σ rule) over recent baselines to identify deviations. When performance degradation is detected, the system leverages the existing three-stage composition pipeline with additional context about current performance characteristics and failure patterns. The monitoring module transmits composition identifiers alongside objective failure evidence to the recomposition endpoint of the orchestrator. This enables the generation of alternative compositions that address identified issues while maintaining functional requirements. The original requirements are augmented with the performance context, allowing the LLM to reason about both old and new constraints simultaneously.

D. User Interface and Visualization

Understanding and validating automated compositions requires effective visualization of service relationships and data flows. OrchestML's web interface presents composition blueprints as interactive directed graphs, where nodes represents services and edges represent data dependencies. Each node also contains valuable information such as service description, source repository, and input/output specifications. The interface supports a complete deployment workflow, allowing users to review generated alternatives, confirm selections, and deploy compositions to production environments. Users can compare multiple composition alternatives through an interactive selector, then proceed through a confirmation panel that displays blueprint summaries, service pipeline details, and deployment configuration options. Once confirmed, compositions are registered with unique identifiers and can be monitored for performance degradation that triggers automatic recomposition.

III. Case Study

To demonstrate OrchestML's end-to-end workflow, we conducted an early investigation of OrchestML in a controlled robotics scenario. OrchestML is tasked to create and maintain an autonomous indoor delivery system that processes voice commands, identifies named recipients, and navigates safely in a dynamic environment.

A. Step 1 - Composition Creation

OrchestML's requirement analysis decomposed the user request into three tasks: speech processing, person identification, and navigation. It then used service retrieval to query through the repository module and discovered *vosrosk* from GitHub[4], *face_recog* from GitHub[5], and *Navigation2 with Dynamic*

[3]https://github.com/timescale/timescaledb

[4]https://github.com/bob-ros2/voskros

[5]https://github.com/elpidiovaldez/face_recog/

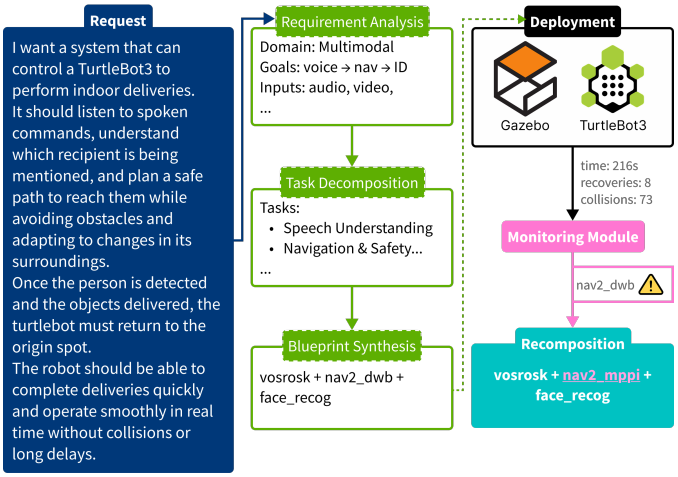
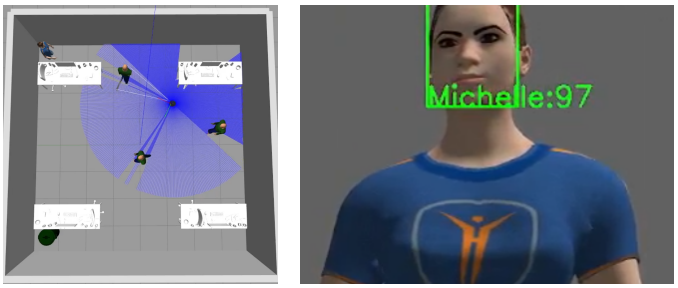


Fig. 3: Overall flow of the case study.

Window Based (DWB) Controller from ROS Index[6]. Blueprint synthesis integrated these into an executable composition graph (Fig. 3).

B. Step 2 - Deployment

The composition was deployed in a TurtleBot3 Gazebo simulation of a square office world with three non-player characters moving randomly (Fig. 4a). The Turtlebot was tasked to deliver from one point to another, stop when it detected the intended person (Fig. 4b), and then return to the start. Minimal middleware was added to bridge ROS topics and enable data exchange between the selected services. The system was executed five times to establish a performance baseline and collect deployment data for a possible recomposition. All three services were exercised in every run.



(a) (b)

Fig. 4: (a) Simulated office world in Gazebo with multiple moving NPCs. "Michelle" is situated at the top left corner of the map. (b) "Michelle" being recognized by the *face_recog* service.

Navigation with DWB completed four runs between 116-158s (baseline mean 140s). One run degraded to 216s, a +55% increase over baseline, with collision (+82%) and recoveries (+127%) also above thresholds. The monitoring module flagged the anomaly and triggered recomposition.

[6]https://index.ros.org/p/nav2_dwb_controller/

C. Step 3 - Recomposition

The orchestrator re-ran the composition procedure with the new information provided by the monitoring module and identified the *Navigation2 with Model Predictive Path Integral (MPPI) Controller* from ROS Index[7] as a replacement. A new blueprint was generated, preserving speech and identification services while substituting MPPI for navigation. Validation runs in the simulated office world with the new MPPI service completed with 109–113 s, representing a 20% improvement over the DWB baseline and a 49% reduction compared to the degraded run.

These results provide early evidence that continuous adaptation of ML compositions is feasible.

IV. Conclusion

We present OrchestML, a prototype tool that composes and maintains ML service compositions across heterogeneous repositories. By combining semantic service discovery, staged blueprint synthesis, and performance-driven recomposition, OrchestML enables continuous adaptation of ML compositions automatically. Our case study in robotic navigation showed that the framework can detect performance degradation and automatically improve composition efficiency, demonstrating the feasibility of closed-loop adaptation.

As future work, we aim to broaden monitoring strategies, explore richer inter-service dependencies, and evaluate scalability across domains beyond robotics.

V. Acknowledgment

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grants funded by the Korea government (MSIT) (RS-2024-00406245, 50% and RS-2025-02218761, 50%).

References

[1] R. Nazir, A. Bucaioni, and P. Pelliccione, "Architecting ML-enabled systems: Challenges, best practices, and design decisions," *Journal of Systems and Software*, vol. 207, p. 111860, Jan. 2024.

[2] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, and T. Azumi, "Autoware on Board: Enabling Autonomous Vehicles with Embedded Systems," in *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*, Apr. 2018, pp. 287–296.

[3] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face," *Advances in Neural Information Processing Systems*, vol. 36, pp. 38 154–38 180, Dec. 2023.

[4] J. Xu, W. Du, X. Liu, and X. Li, "LLM4Workflow: An LLM-based Automated Workflow Model Generation Tool," in *2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Oct. 2024, pp. 2394–2398, iSSN: 2643-1572.

[5] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, Dec. 2022.

[6] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in *The Eleventh International Conference on Learning Representations*, Sep. 2022.

[7] L. Weng, "Llm-powered autonomous agents," *lilianweng.github.io*, Jun 2023. [Online]. Available: https://lilianweng.github.io/posts/2023-06-23-agent/

[7]https://index.ros.org/p/nav2_mppi_controller/